Assignment 4: Relational Database vs. NoSQL Database

Comparative report on scalability, consistency, performance and use cases

1. Introduction

A **relational database (RDBMS)** stores data in tables with a fixed schema, links tables through keys, and is queried with SQL (MySQL, PostgreSQL, Oracle, SQL Server). A **NoSQL** database is any non-relational system built for flexible schemas and horizontal scale. NoSQL is not one technology but four main families. Neither is universally better: the right choice depends on the data and the workload.

2. Types of NoSQL Databases

Type	How data is stored	Examples	Typical use
Document	JSON-like documents grouped in collections	MongoDB, Couchbase	Product catalogs, content management, user profiles, clinical documents
Key-value	Key mapped to an opaque value	Redis, Amazon DynamoDB	Caching, sessions, shopping carts, leaderboards
Wide-column	Rows with flexible column families, partitioned across nodes	Apache Cassandra, HBase	Time-series, event logs, IoT, very high write volumes
Graph	Nodes and edges	Neo4j, Amazon Neptune	Fraud detection, recommendations, social/knowledge networks

3. Comparison

Criterion	Relational (SQL)	NoSQL
Data model and schema	Tables, rows, columns; schema defined up front (schema-on-write); strong typing and constraints.	Documents, key-value pairs, column families or graphs; flexible or dynamic schema (schema-on-read); fields can differ per record.
Scalability	Traditionally scales **vertically** (bigger server). Read replicas help reads; **sharding** is possible but complicated because joins and multi-row transactions span shards. Distributed SQL (CockroachDB, Google Spanner) narrows the gap.	Designed to scale **horizontally** by adding commodity nodes, with automatic partitioning (sharding) and replication built in. Capacity and throughput grow roughly with node count.
Consistency	**ACID** transactions: atomic, consistent, isolated, durable. Strong consistency is the default and constraints (PK, FK, CHECK) protect integrity.	Many systems follow **BASE** (basically available, soft state, eventually consistent) and favour availability and partition tolerance (CAP theorem). Consistency is often tunable (Cassandra consistency levels, DynamoDB strongly-consistent reads); MongoDB supports multi-document ACID transactions.
Performance	Excellent for complex queries: joins, aggregates, ad-hoc reporting, with a cost-based optimizer and indexes. Heavy joins on huge, sharded data become slow.	Excellent for simple, high-volume reads/writes by key with predictable low latency (data is denormalized so a query touches one record/partition). Weak at ad-hoc joins and cross-entity aggregation.
Query language	Standard SQL - portable and widely known.	Varies: MongoDB Query Language, CQL (Cassandra), Cypher (Neo4j), simple get/put APIs.
Relationships	First-class: foreign keys and joins; referential integrity enforced.	Embedded documents or application-managed references; no enforced referential integrity (graph databases are the exception).
Data redundancy	Normalized to minimize duplication.	Deliberately denormalized/duplicated to make reads fast; the application must keep copies in sync.
Maturity and tooling	Decades of tools, standards, skills, BI/reporting support.	Newer; tooling and skills vary by product; fewer standards.
Best suited for	Structured data with clear relationships and strict correctness: banking, ERP, inventory, billing, hospital core records.	Massive, fast-changing or unstructured data and global scale: social feeds, IoT telemetry, catalogs, caching, real-time analytics.

4. Why Organizations Choose Different Database Technologies

Organizations choose a database by matching its strengths to their requirements, not by fashion. The main decision factors are:

- **Nature of the data** - tabular and relational data fits SQL; nested, varied or unstructured data fits documents; connected data fits graphs.
- **Correctness requirements** - money transfers, patient records and inventory need ACID guarantees; a 'likes' counter can tolerate eventual consistency.
- **Scale and traffic** - a single well-tuned RDBMS serves most businesses; internet-scale write volumes or multi-region availability push towards NoSQL.
- **Query patterns** - ad-hoc reporting and joins favour SQL; a few known, key-based access patterns favour NoSQL.
- **Rate of schema change** - rapidly evolving products benefit from flexible schemas; regulated systems benefit from strict ones.
- **Availability and latency targets** - always-on, geo-distributed services may trade strict consistency for uptime.
- **Cost, skills and ecosystem** - licensing, cloud managed services, team expertise, and integration with existing tools.
- **Compliance** - audit trails, access control, and data-integrity rules (healthcare, finance) often favour mature relational systems.

In practice many organizations use **polyglot persistence**: several databases, each used for what it does best, for example a relational database for transactions, a document store for content, and Redis for caching.

5. Real-World Examples

The examples reflect publicly documented usage patterns; companies' stacks change over time, so treat them as illustrations.

Organization / domain	Database used for the workload	Why it fits
Banks and payment systems (Oracle, IBM Db2, PostgreSQL, SQL Server)	Accounts, ledgers, transfers	Strict ACID transactions: a transfer must debit and credit atomically; strong integrity and audit needs.
Instagram (PostgreSQL)	Core user and relationship data	Structured data with relationships; mature tooling and strong consistency, scaled with replication and sharding.
Netflix (Apache Cassandra)	Viewing history and other high-volume, globally distributed data	Very high write throughput, multi-region replication and availability with tunable consistency.
Amazon (DynamoDB, descended from the Dynamo design)	Shopping cart, sessions, key-based lookups	Predictable single-digit-millisecond key access at huge scale; availability prioritized.
E-commerce and content platforms (MongoDB)	Product catalogs, CMS, user profiles	Items have different attributes; a flexible document maps naturally to a JSON product page.
Many web applications (Redis)	Caching, sessions, rate limiting, leaderboards	In-memory key-value access is extremely fast.
Fraud detection and recommendations (Neo4j)	Relationship traversal across accounts, devices, people	Graph queries follow many hops efficiently, where SQL would need many self-joins.

6. Application to the Hospital Management System

Our hospital database (patients, doctors, appointments, admissions, bills, prescriptions) is **relational by nature**: entities are highly connected through foreign keys, billing and prescriptions require transactional correctness, and administrators need joins and aggregate reports. A hospital could still add NoSQL components where they fit better:

Hospital requirement	Better fit	Reason
Patients, appointments, admissions, billing, prescriptions, stock	Relational (MySQL/PostgreSQL)	ACID, foreign keys, constraints, reporting joins
Continuous ICU/wearable vital-sign streams	Wide-column or time-series (Cassandra)	Massive append-only writes, time-ordered queries

Hospital requirement	Better fit	Reason
Unstructured clinical notes, imaging metadata, FHIR/JSON documents	Document store (MongoDB)	Flexible structure per document
Login sessions, dashboards, frequently read lookups	Key-value cache (Redis)	Very low latency
Disease-contact tracing, doctor-patient referral networks	Graph (Neo4j)	Multi-hop relationship queries

7. Conclusion

Relational databases provide strong consistency, a rich query language and integrity guarantees, which makes them the default for transactional, structured systems. NoSQL databases provide schema flexibility, horizontal scalability and high throughput for large or loosely structured data, usually in exchange for weaker consistency guarantees or limited joins. Organizations choose according to data structure, consistency needs, scale, query patterns and cost, and large systems commonly combine both. For our Hospital Management System the relational model is the correct core choice, with NoSQL as an optional complement for specialised workloads.